

Iceberg Bronze-Layer Ingestion Pipeline

Spark + Nessie + Iceberg + MinIO — Technical Documentation

Prepared for internal engineering reference

Document version 1.0

1. Overview

- This document describes a PySpark ingestion pipeline that loads two CSV sources — an internal company sales feed and a scraped competitor dataset — deduplicates them, and upserts them into Apache Iceberg tables managed through a Project Nessie catalog, with data stored in MinIO (S3-compatible object storage).
- The pipeline follows a write-audit-publish (WAP) pattern using Nessie's Git-like branching: all writes land on an isolated branch first, are validated, and are only merged into the main branch once validation passes. This isolates in-progress writes from any process reading the main branch concurrently.

1.1 Technology stack

Component	Role
Apache Spark 3.3 (PySpark)	Compute engine executing all reads, dedup logic, and writes
Apache Iceberg 1.3.1	Table format providing schema evolution, MERGE INTO, and time travel
Project Nessie 0.67.0 (client)	Git-like catalog: branches, commits, merges for atomic multi-table changes
MinIO	S3-compatible object storage backing the Iceberg warehouse

1.2 Source and target objects

Source file	Target Iceberg table	Composite key
raw_company_system.csv	nessie.bronze.raw_company_system	Store_Name, Transaction_ID, Store_ID
raw_competitor_scraper.csv	nessie.bronze.raw_competitor_scraper	Scraped_At, Competitor_Name, Product_ID

2. High-level execution flow

Functions	Role
run_pipeline()	Is the single entry point. It executes all functions
spark_config()	Builds the SparkConf (Iceberg + Nessie packages, catalog config, MinIO credentials).
build_app()	Instantiates the SparkSession and suppresses INFO/WARN logging.
read_data()	Loads both CSVs and deduplicates each via remove_duplicate().
ingest_and_validate()	The WAP core: branch, guardrail check, upsert, integrity check.
merge_to_main()	Merges the isolation branch into main and drops it.

A rendered flowchart of this execution path, including every decision branch and error-handling path

3. Function reference

3.1 spark_config()

Builds and returns a SparkConf configured with: Iceberg and Nessie Spark extension packages, a SparkCatalog named “nessie” pointed at the Nessie REST URI (from the NESSIE_URI environment variable), MinIO credentials (from MINIO_ACCESS_KEY / MINIO_SECRET_KEY), and catalog caching explicitly disabled.

Configuration keys set

Key	Value / source	Purpose
spark.sql.catalog.nessie.uri	env: NESSIE_URI	Nessie REST endpoint
spark.sql.catalog.nessie.ref	"main"	Default branch at session start
spark.sql.catalog.nessie.warehouse	s3a://warehouse	Iceberg data/metadata root in MinIO
spark.sql.catalog.nessie.cache-enabled	"false"	Disables Iceberg CachingCatalog
spark.hadoop.fs.s3a.access.key / secret.key	env: MINIO_ACCESS_KEY / MINIO_SECRET_KEY	MinIO S3 credentials

3.2 build_app(conf)

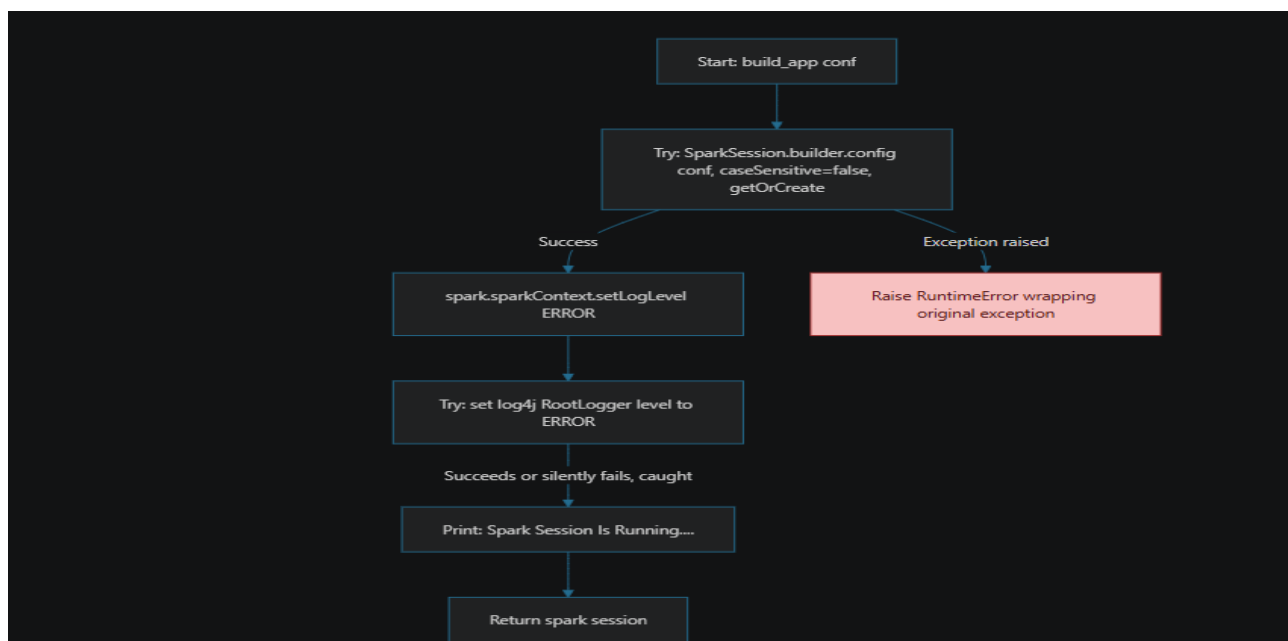
Initializes and configures a PySpark Session with optimized log management. This function sets up a SparkSession using the provided configuration, enforces case-insensitive column resolution for SQL queries, and suppresses non-critical logs (INFO/WARN) to clean up terminal output.

Parameters	Descriptions
conf : SparkConf	A PySpark SparkConf object containing custom configuration settings for the session (catalog, warehouse, and storage credentials).

Returns: Spark Session An active, configured PySpark Spark Session instance.

Raises: Runtime Error If the Spark Session fails to initialize, wrapping the underlying exception.

Internal execution flow



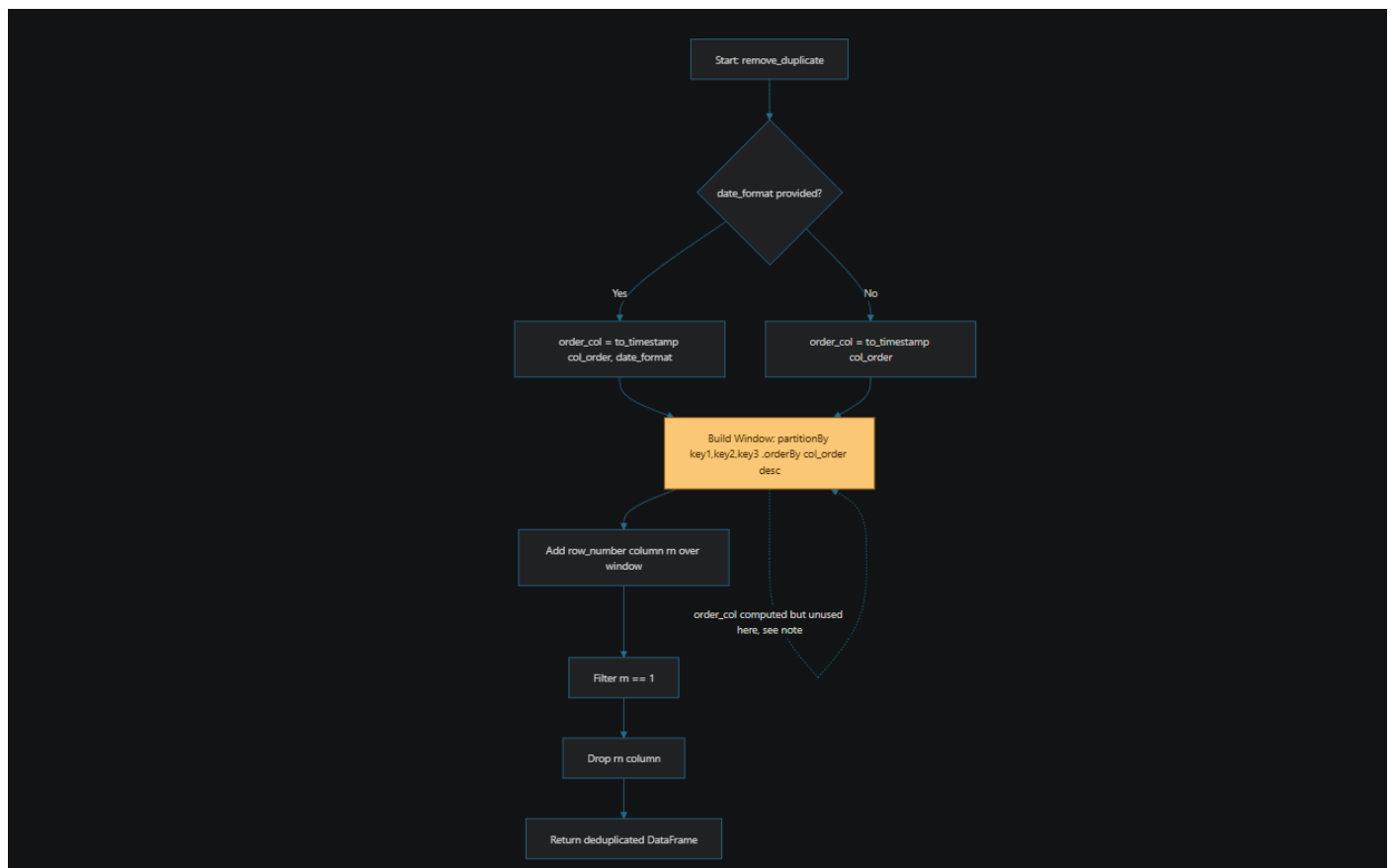
3.3 remove_duplicate(df, key1, key2, key3, col_order, date_format=None)

Deduplicates a DataFrame by keeping only the most recent record per composite key. Uses a PySpark Window function (row_number) partitioned by a 3-column composite key and ordered descendingly by a timestamp/date column to isolate the latest row per key group.

Parameters	Descriptions
df : DataFrame	The input PySpark DataFrame containing potential duplicate records.
key1 : str	The first column name of the composite primary/partition key.
key2 : str	The second column name of the composite primary/partition key.
key3 : str	The third column name of the composite primary/partition key.
col_order : str	The column name used to determine record recency (e.g., date or timestamp).
date_format : str, optional	Custom date format string for parsing `col_order` via `to_timestamp` (default is None).

Returns: DataFrame A deduplicated PySpark DataFrame containing only the latest row for each key combination.

Internal execution flow



3.4 read_data(app)

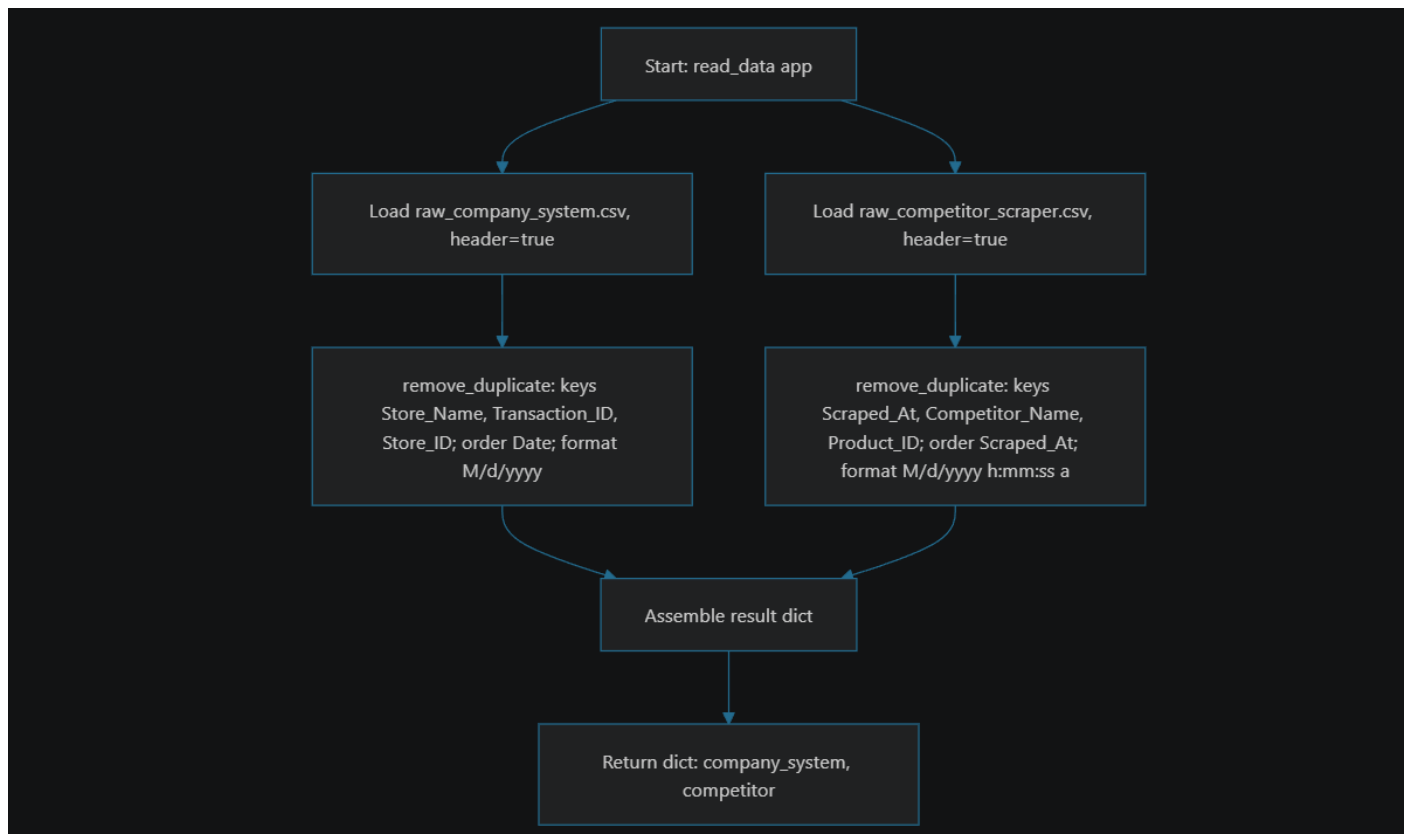
Reads raw CSV datasets and applies domain-specific deduplication. Loads company system transactions and scraped competitor product data from CSV files, then purges duplicates using `'remove_duplicate'` based on each dataset's respective composite key and date format.

Returns: dict of {str: DataFrame}

- A dictionary containing cleaned and deduplicated PySpark DataFrames:

- `'company_system'`: Deduplicated internal sales/store transactions.
- `'competitor'`: Deduplicated scraped market/competitor data.

Internal execution flow



3.5 upsert_to_iceberg_table(df, target_table, key1, key2, key3, temp_view_name, branch_name, app)

- Executes a smart upsert operation into an Apache Iceberg table.

- This function handles two operational paths, determined by a branch-qualified DESCRIBE existence check:

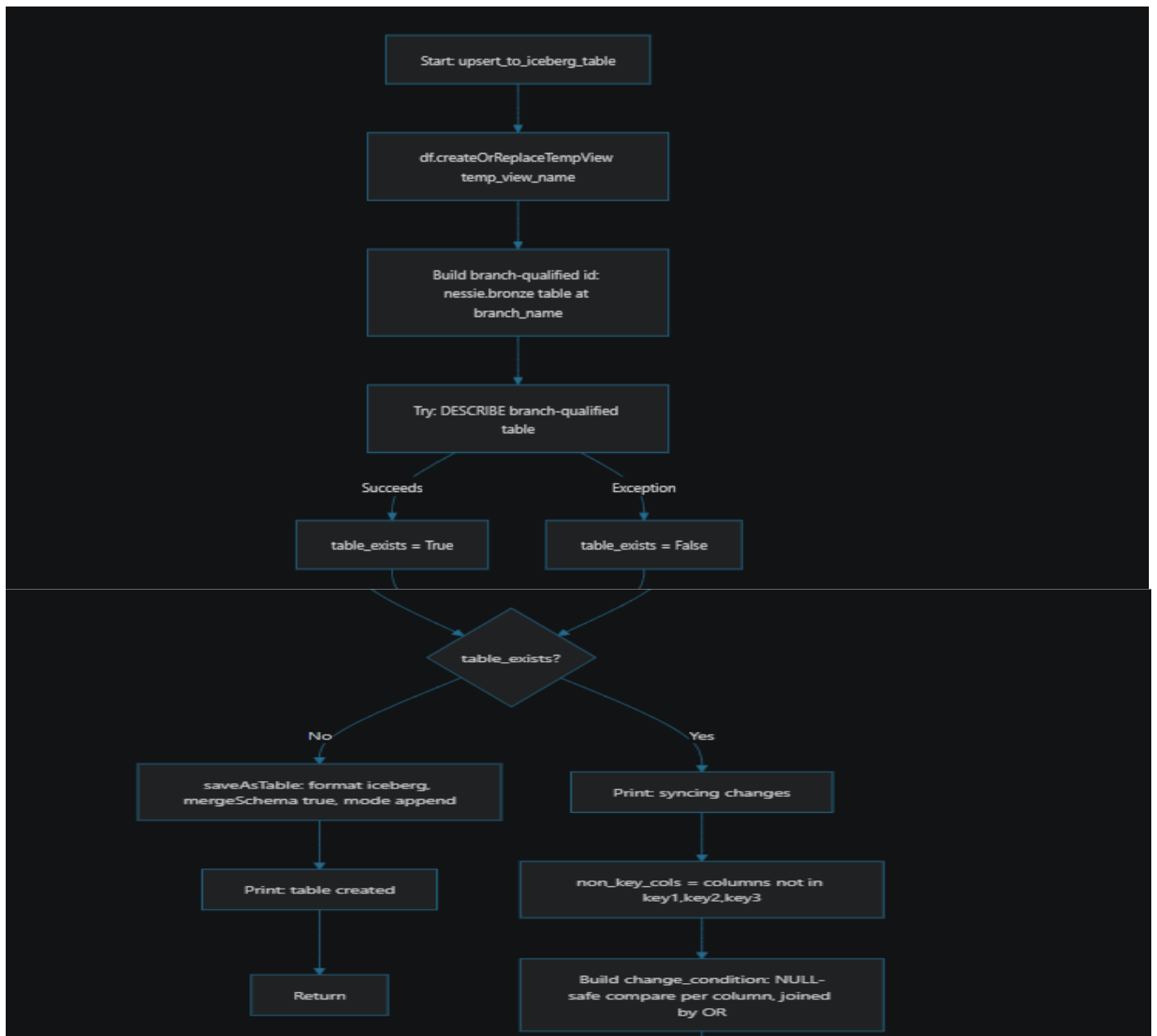
1. Cold start: creates the target table and loads the initial batch if it does not yet exist on the active branch.
2. Conditional MERGE: executes a NULL-safe MERGE INTO query for updates and inserts on changed

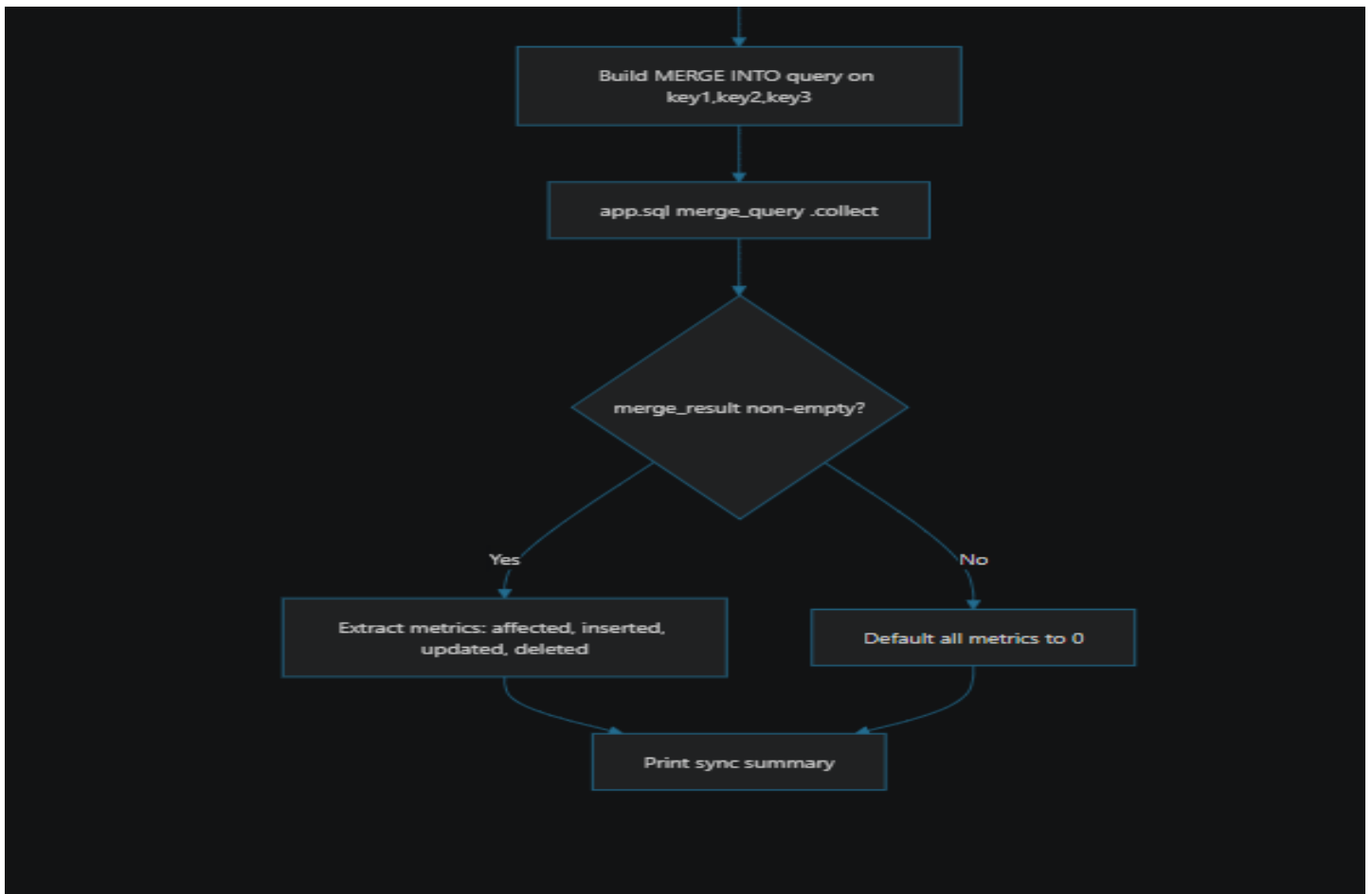
rows,

then reports the resulting row-level metrics (affected / inserted / updated / deleted).

Parameters	Descriptions
df : DataFrame	Incoming Spark DataFrame to be ingested
target_table (str)	Name of the destination Iceberg table in the Nessie catalog.
key1 : str	The first column name of the composite primary/partition key.
key2 : str	The second column name of the composite primary/partition key.
key3 : str	The third column name of the composite primary/partition key.
temp_view_name (str)	Temporary view name used for staging SQL execution.
branch_name (str)	Active Nessie isolation branch context.
app (SparkSession)	Active Spark session with Nessie SQL capabilities.

Internal execution flow





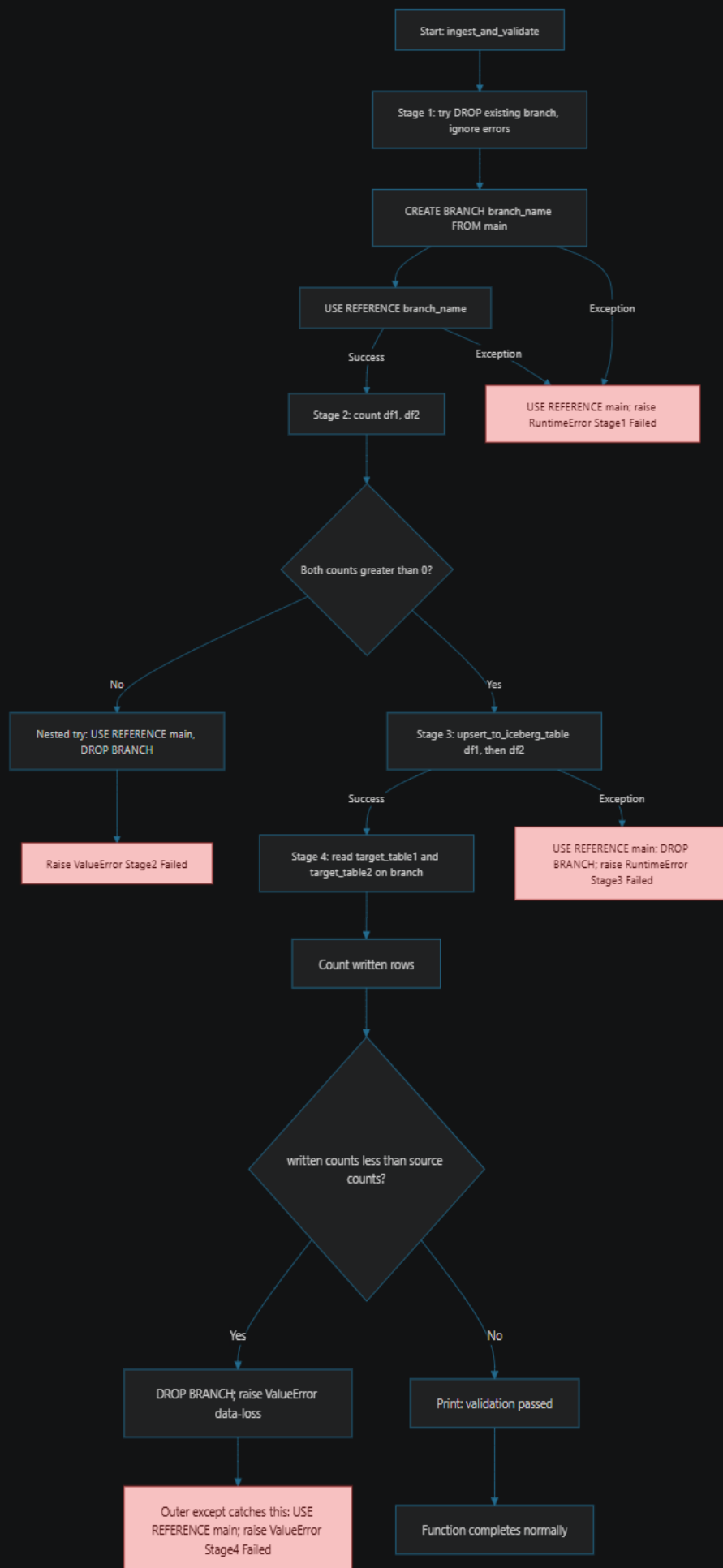
3.6 ingest_and_validate(df1, df2, branch_name, target_table1, target_table2, app)

Orchestrates four stages, each wrapped in its own try/except that raises a stage-specific error on failure:

- Stage 1 — Branch provisioning: drops any pre-existing branch of the same name (best-effort, exceptions swallowed), creates a fresh branch from main, and switches the session to it.
- Stage 2 — Guardrail check: counts both source DataFrames and aborts if either is empty. Cleans up the branch before raising.
- Stage 3 — Upsert: calls `upsert_to_iceberg_table()` for each table. Cleans up the branch before raising on any failure.
- Stage 4 — Integrity check: reads both tables back (still scoped to the isolation branch via the active session reference) and compares row counts against the pre-write source counts. If either target has fewer rows than its source, drops the branch and raises. for an asymmetry in this stage's cleanup path.

Parameters	Descriptions
df 1: DataFrame	Spark DataFrame containing company source data.
df 2: DataFrame	Spark DataFrame containing competitor source data.
target_table1 (str)	Target Iceberg table for df2.
target_table2 (str)	Target Iceberg table for df2.
branch_name (str)	The isolated Nessie branch name for staging changes.
app (SparkSession)	Active Spark session with Nessie SQL capabilities.

Internal execution flow



3.7 merge_to_main(branch_name, app)

- Merges an isolated staging branch back into the 'main' branch in Project Nessie and performs automated branch cleanup.

- This function executes an atomic merge operation on the Nessie catalog, incorporating all staging data and schema modifications into the live production state ('main'). Upon a successful merge, it switches the context back to 'main' and safely drops the temporary staging branch.

3.8 run_pipeline()

Top-level orchestrator. Wraps the entire sequence (config → session → read → ingest_and_validate → merge_to_main) in a single try/except Exception that prints the error and does not re-raise.

Master end-to-end flowchart



4. Required environment variables

Variable	Consumed by	Example
NESSIE_URI	spark_config()	http://nessie:19120/api/v2
MINIO_ACCESS_KEY	spark_config()	admin
MINIO_SECRET_KEY	spark_config()	admin12345

All three must be present in the environment the Spark driver process actually runs in (e.g. set at the container level, not just in a .env file that nothing sources).

5. Known limitations and recommendations

These are called out explicitly because several are not visible from a successful test run and would surface later, either as silent data-quality issues or as orchestration blind spots.

5.1 Catalog caching — correctly disabled, worth keeping that way

cache-enabled is set to "false", which is correct given this pipeline switches Nessie branches within a single long-lived SparkSession. Iceberg's default CachingCatalog keys cached Table handles by identifier only, with no awareness of which branch was active when the handle was cached — re-enabling caching here can silently reintroduce stale reads across branch switches.

5.2 remove_duplicate(): parsed timestamp is computed but not used for ordering

order_col = to_timestamp(col(col_order), date_format) is computed but the Window's .orderBy(col(col_order).desc()) sorts on the original, unparsed string column instead of order_col. For non-lexicographically-sortable date formats (e.g. "M/d/yyyy", where "12/1/2023" sorts before "2/1/2023" as a string), this can select the wrong row as "latest" within a duplicate group. Recommendation: sort on order_col, not col_order.

5.3 Change-condition parenthesization

t.{col} <=> s.{col} = FALSE relies on <=> and = sharing the intended precedence in Spark SQL's parser. The unambiguous equivalent, NOT (t.{col} <=> s.{col}), removes any dependency on operator precedence and is recommended regardless of whether the current form parses as intended.

5.4 Stage 4 cleanup asymmetry

Within Stage 4, the explicit row-count check drops the branch before raising. However, the surrounding except block (which catches any other exception in Stage 4, e.g. a read failure) only issues USE REFERENCE main — it does not drop the branch. Any non-count-related failure in Stage 4 leaves an orphaned branch behind. Recommendation: drop the branch unconditionally in Stage 4's except block, guarded by its own try/except so a failed cleanup doesn't mask the original error.

5.5 merge_to_main(): no cleanup on merge failure

If MERGE BRANCH ... INTO main fails, the exception is printed and re-raised, but the branch is never dropped and the session reference is never reset to main. The branch is left dangling and requires manual cleanup (or will be picked up, confusingly, by the next run's pre-emptive DROP BRANCH in Stage 1).

5.6 run_pipeline(): exceptions are swallowed, not propagated

This is the most operationally significant item in this document. The outer except Exception as pipeline_error block prints the error and returns normally — it does not re-raise, and the script does not call sys.exit() with a non-zero code. A process orchestrator (cron, a shell wrapper, or notably Airflow via a PythonOperator / BashOperator / SparkSubmitOperator) determines task success or failure from the process's exit status or an unhandled exception, not from stdout text. As written, every failure mode documented above — a failed branch creation, an empty source guardrail trip, an upsert failure, a data-loss integrity failure — will be logged to stdout but reported to the orchestrator as a successful run. Recommendation: re-raise pipeline_error after logging (or call sys.exit(1)) so failures are visible to whatever schedules this pipeline.

5.7 Static branch name reused across runs

branch_name = "ingest_dev" is a fixed literal. Two overlapping runs (or a retry of a run that's still cleaning up) will race on the same branch name. Recommendation: derive the branch name per run — e.g. from an orchestrator's run ID, or a UUID/timestamp — so concurrent or retried executions are isolated from one another.

5.8 No defensive dedup inside upsert_to_iceberg_table()

The function trusts that df arrives pre-deduplicated by read_data() (the dropDuplicates() call inside upsert_to_iceberg_table() itself is commented out). This is consistent today, since the composite keys used in read_data()'s remove_duplicate() calls match the merge keys used here — but the function has no guardrail if that

invariant is ever broken by a future caller. A batch with duplicate keys will cause Iceberg's MERGE INTO to fail ("multiple source rows matched") or behave inconsistently.

5.9 Competitor dedup key includes its own order column

The competitor dedup call partitions on (Scraped_At, Competitor_Name, Product_ID) and also orders by Scraped_At. Since Scraped_At is part of the partition key, every row within a given partition shares the same Scraped_At value by definition — the ordering therefore cannot differentiate “latest” among true duplicates, and row_number() effectively picks an arbitrary row among exact ties. Worth confirming whether this is intentional (i.e. Scraped_At granularity is fine-grained enough that true duplicates are rare) or whether a different tiebreaker column was intended.

6. Final Outputs

```
===== Data Ingestion Execution Started =====
26/07/22 09:22:22 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.
Spark Session Is Running...
===== Ingest AND Validate Execution Started =====
1- Creating & Switching to isolation branch: ingest_dev
  └─ Creating & Switching Is: DONE!
-----
2- Checking source datasets for data presence...

  └─ Source data validation passed successfully:
      └─ Company Data : 50,000 rows found
      └─ Competitor Data: 13,291 rows found
-----
3- Writing data to Iceberg table and Executing Incremental Upsert to Iceberg tables...
--- New records detected! Syncing changes to 'nessie.bronze.raw_company_system' (Updating existing & Adding new)...

Data sync completed successfully for 'nessie.bronze.raw_company_system':
└─ Affected (Changed) : 0
└─ Inserted (New Rows) : 0
└─ Deleted (Changed) : 0
└─ Updated (Changed) : 0
--- New records detected! Syncing changes to 'nessie.bronze.raw_competitor_scraper' (Updating existing & Adding new)...

Data sync completed successfully for 'nessie.bronze.raw_competitor_scraper':
└─ Affected (Changed) : 0
└─ Inserted (New Rows) : 0
└─ Deleted (Changed) : 0
└─ Updated (Changed) : 0
  └─ Data successfully written to isolation branch!
-----
4- Performing Atomic Writes Validation Checks...
Target Company Table Total Rows on 'ingest_dev': 50000
Target Competitor Table Total Rows on 'ingest_dev': 13291
Validation Passed: Row counts and Schema Evolution are completely verified!
===== Merging Branch Execution Started =====
Merging branch 'ingest_dev' into 'main'...
PIPELINE SUCCESSFUL: Data & Schema changes are now live on 'main'!
===== Data Ingestion Execution Finished Successfully =====
```